

Базы данных

Тема 12

Параллельные и распределенные базы данных

Нефункциональные требования

- производительность:
 - пропускная способность (*throughput*) – количество действий за единицу времени
 - время отклика – время выполнения одного действия;
- масштабируемость (*scalability*) – определяет изменения какой-либо характеристики производительности при изменении нагрузки (количества пользователей, объёма накапливаемой информации) и размеров вычислительной системы;
- надежность (*reliability*) и отказоустойчивость (*fault tolerance*);
- доступность (*availability*): отношение времени нормальной работы системы (возможность выполнения запросов на чтение и/или запись) к длине интервала времени измерения.

Параллелизм

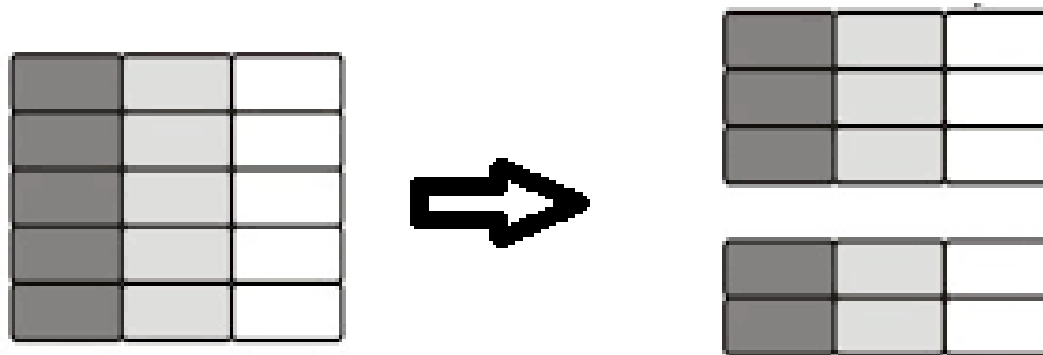
- гранулярность параллелизма:
 - между транзакциями;
 - между запросами внутри транзакций;
 - между операциями плана выполнения внутри запросов;
 - в алгоритмах выполнения операций;
- издержки:
 - синхронизация между параллельно выполняемыми операциями с данными;
 - проблема равномерного распределения нагрузки;
- преимущества: высокая масштабируемость для отдельных типов операций.

Распределение данных

- на физическом уровне (блоки данных):
 - необходимы программные или аппаратные средства управления размещением данных на дисках, фактически замещающие функции файловой системы ОС (например, RAID или распределённые файловые системы с поддержкой интерфейса блочного доступа);
- на логическом уровне (строки/столбцы таблиц и сами таблицы)
 - горизонтальная фрагментация / секционирование (*horizontal partitioning*);
 - вертикальная фрагментация / секционирование (*vertical partitioning*);
 - репликация / тиражирование (*replication*).

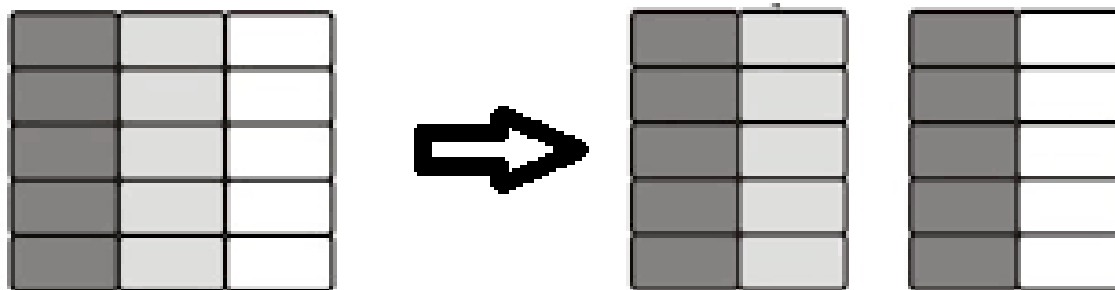
Горизонтальная фрагментация

- горизонтальная фрагментация – разбиение таблицы на совокупность попарно непересекающихся секций (*partition*) по значениям *ключа секционирования*;
- разбиение может определяться:
 - диапазонами значений ключа секционирования;
 - значениями функции хеширования, вычисляемыми на основе ключа секционирования;
 - списками значений ключа секционирования.



Вертикальная фрагментация

- вертикальная фрагментация – разбиение таблицы на части таким образом, чтобы каждая часть содержала только некоторые из столбцов исходной таблицы (разбиение строк);
- каждая из частей содержит первичный ключ, обеспечивающий возможность соединения частей для получения значений полного набора атрибутов.



Репликация

- репликация – создание множественных копий данных (таблиц);
- процедура репликации должна обладать прозрачностью для клиентов СУБД;
- согласованность реплик:
 - строгая (глобальная) согласованность: единая логическая копия;
 - согласованность на уровне отдельных транзакций;
 - локальная согласованность (например, разделение реплик для оперативной и аналитической обработки);
- согласованность реплик определяют правила обновления и распространения изменений по репликам.

Поддержка единой логической копии

- изменения, вносимые любой транзакцией, должны быть выполнены на всех серверах до того, как транзакция будет зафиксирована;
- методы реализации:
 - глобальные транзакции;
 - главная копия: все обновляющие транзакции выполняются на главном (*primary*) сервере, и внесённые изменения до фиксации распространяются на остальные копии, расположенные на резервных (*standby*) серверах;
- недостаток: снижается доступность (как минимум, для записи) и производительность системы.

Репликация главной копии

- различные методы распространения обновлений, влияющие на:
 - доступность системы;
 - отставание состояния резервных серверов;
 - связанную с репликацией нагрузку на серверы;
- синхронное распространение изменений: до фиксации транзакции, выполнившей изменения на главном сервере;
- асинхронное распространение изменений, предполагающее отставание состояния резервных серверов:
 - (повторение операторов SQL);
 - обновление логических записей;
 - на уровне страниц баз данных;
 - распространение записей журнала.

Репликация главной копии: варианты использования

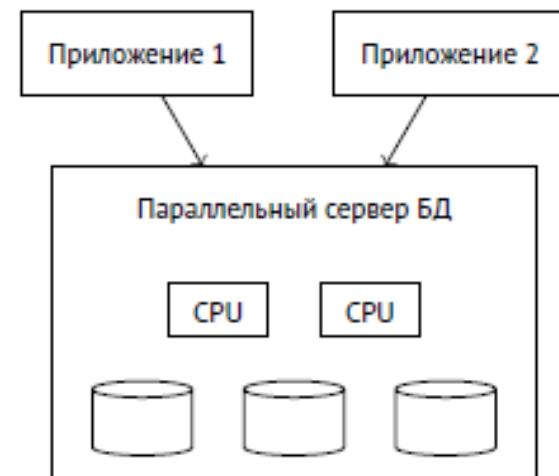
- создание копий, отражающих состояние базы данных на основном сервере на определённый момент времени (без последующего распространения изменений);
- перераспределение нагрузки на главный сервер (перераспределение операций чтения на резервные серверы);
- перераспределение запросов в зависимости от типов нагрузки (оперативная и аналитическая обработка данных);
- создание резервных копий базы данных для переключения в случае отказа.

Масштабируемость

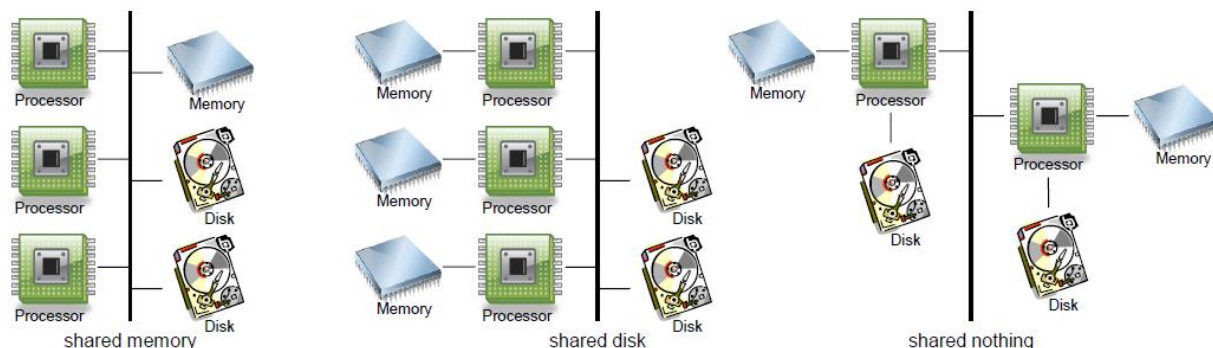
- централизованные / монолитные СУБД
 - реляционные СУБД: Oracle, PostgreSQL, MS SQL Server, ... ;
 - вертикальное масштабирование (*scale-up, vertical scaling*);
- параллельные СУБД (*parallel database systems*):
 - Teradata, Oracle Real Application Cluster, ... ;
 - вертикальное / горизонтальное масштабирование;
- распределенные СУБД (*distributed database systems*);
 - на базе монолитных СУБД / нереляционные СУБД (MongoDB, Cassandra, HBase, Google BigTable, Neo4j, Redis, Memcached, Tarantool,...);
 - горизонтальное масштабирование (*scale-out, horizontal scaling*).

Параллельные СУБД

- основная цель: обеспечение высокой производительности за счет выполнения запроса (транзакции) или отдельных операций на нескольких вычислительных узлах (устройствах);
- с точки зрения клиента не отличается от централизованной СУБД;
- единая схема данных (для каждой базы данных);
- проблема балансировки нагрузок между вычислительными узлами как с точки зрения выполнения запросов, так и с точки зрения распределения хранимых данных (для архитектуры SN из-за неравномерности скорости доступа к носителям данных).



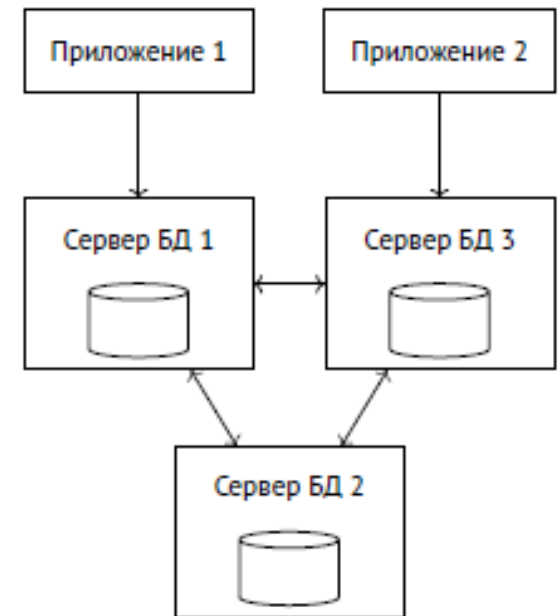
Аппаратная конфигурация параллельных СУБД



- SM (shared memory):
 - многопроцессорные системы с разделяемой памятью;
 - масштабируемость ограничена возможным количеством устанавливаемых процессоров в одной вычислительной системе и пропускной способностью;
- SD (shared disk):
 - вычислительные устройства взаимодействуют между собой и с устройствами дисковой памяти через коммуникационную сеть;
 - масштабируемость ограничена необходимостью синхронизации для предотвращения конфликтующих изменений страниц;
- SN (shared nothing):
 - каждая вычислительная система функционирует автономно и взаимодействие через вычислительную сеть.

Распределённые СУБД

- основные цели:
 - повышение доступности;
 - горизонтальная масштабируемость;
 - интеграция разнородных систем;
- совокупность серверов баз данных, каждый из которых может работать независимо от остальных серверов;
- отдельный сервер поддерживает свою схему данных, но может выполнять запросы к данным, размещённым на нескольких серверах (прозрачный доступ к данным);
- с точки зрения используемых на серверах СУБД могут быть однородными и неоднородными;
- возможные ограничения реализации:
 - доступность некоторых данных;
 - выполнение требований ACID для распределённых транзакций.



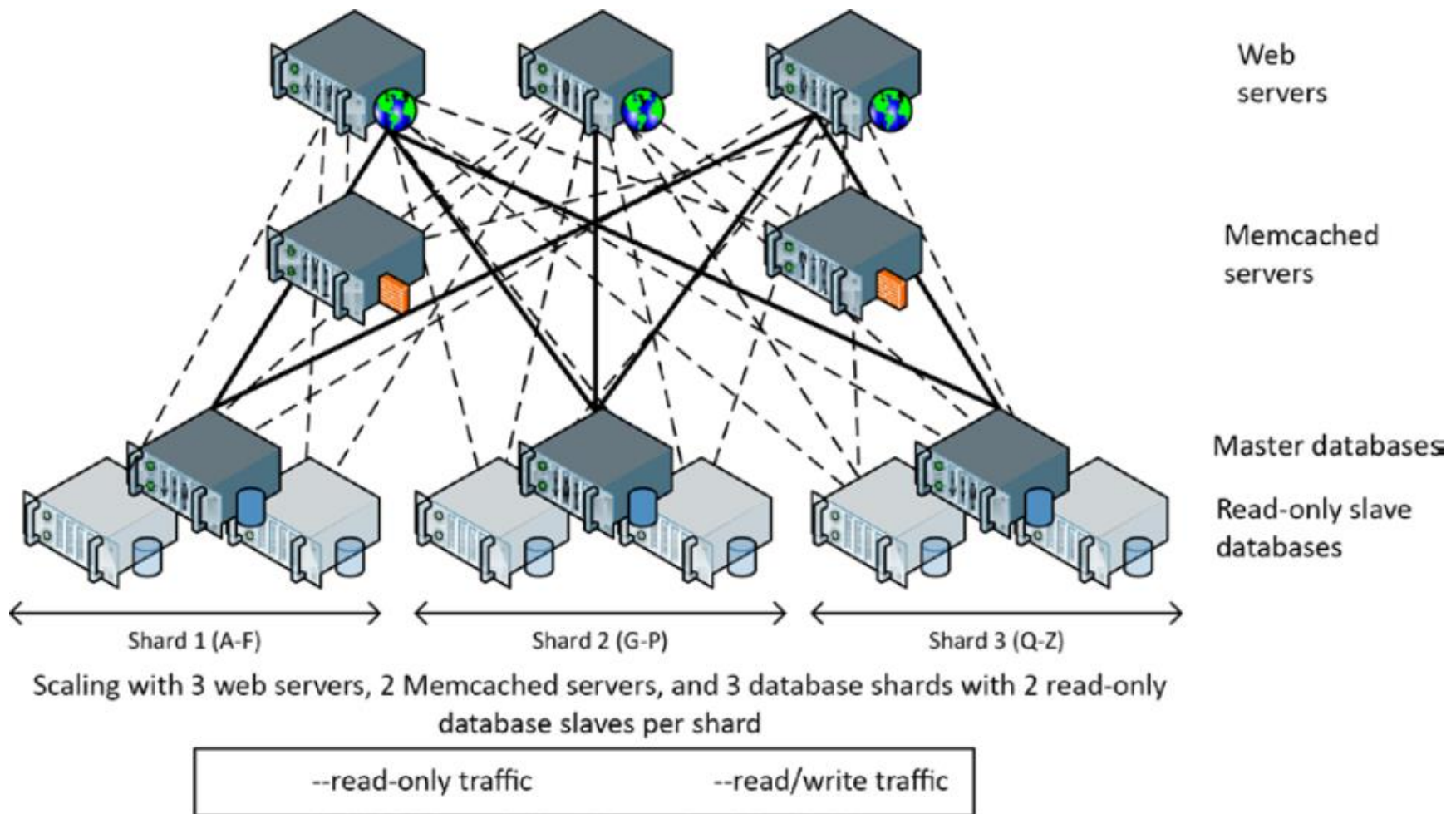
Согласованность в распределённых системах

- глобальные (распределённые) транзакции – содержат операции обработки элементов данных на разных серверах:
 - должны удовлетворять требуемому уровню изоляции;
 - должны завершаться одинаково (фиксацией или откатом);
- локальные транзакции – обрабатывают данные, находящиеся на одном сервере;
- глобальные протоколы управления транзакциями должны обеспечивать высокую доступность, отсутствие централизованных механизмов управления и горизонтальную масштабируемость → ослабленные критерии согласованности в реализациях распределённых систем;
- протоколы на основе меток времени: распределённый протокол изоляции моментальных снимков.

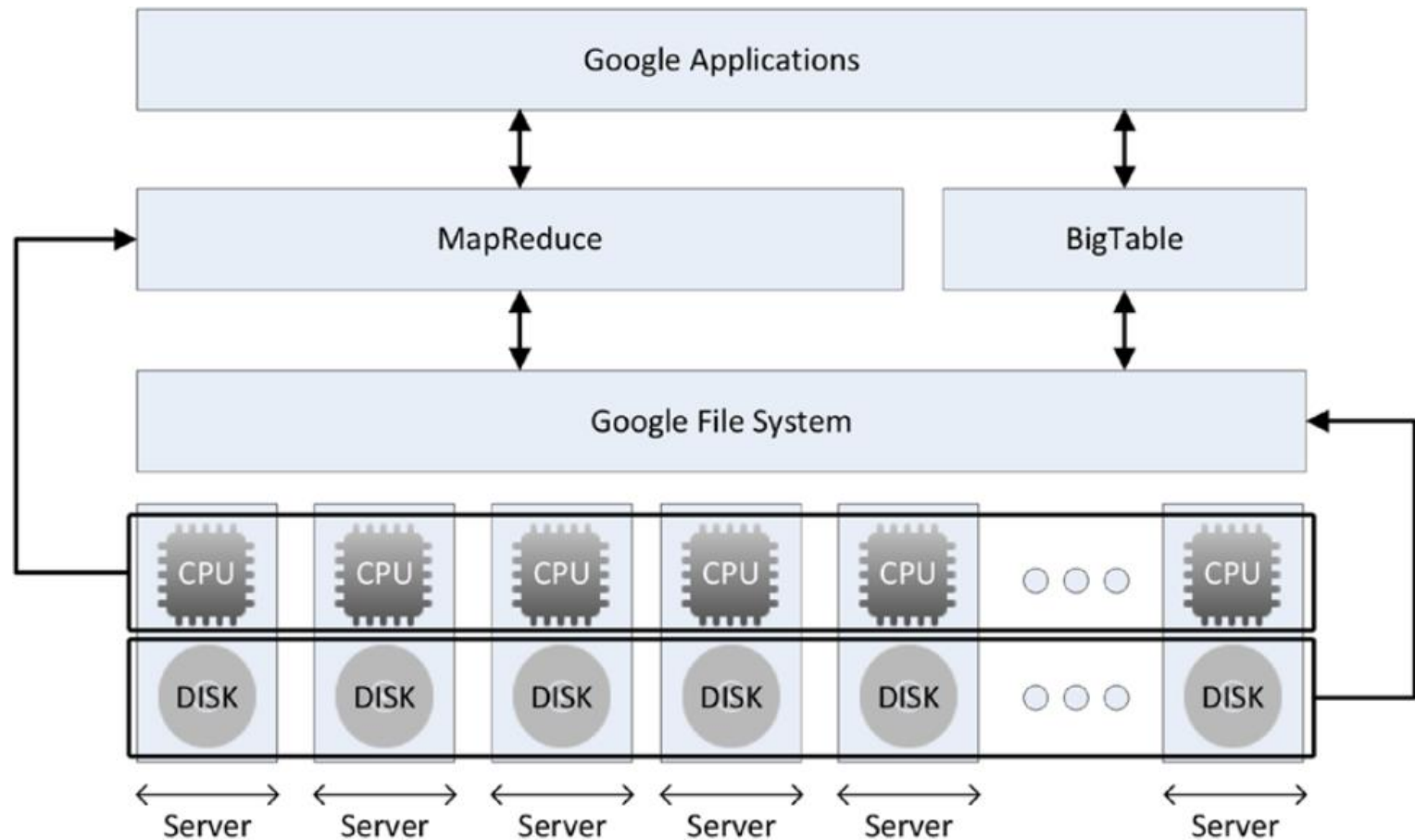
Двухфазная фиксация транзакций

- протокол двухфазной фиксации транзакций (*2PC – two-phase commit protocol*) определяет алгоритм завершения распределённых транзакций
- участники распределенной транзакции:
 - два или более сервера, которые называются *диспетчерами ресурсов*;
 - *диспетчер транзакций* (координатор распределенных транзакций), который обеспечивает управление транзакцией путем координации между диспетчерами ресурсов;
- двухфазная фиксация транзакций:
 - фаза подготовки: каждый из диспетчеров ресурсов обеспечивает устойчивость транзакции, в частности, записывая журнал транзакции на диск;
 - фаза фиксации: начинается в случае успешного завершения подготовки всеми диспетчерами ресурсов и завершается после получения диспетчером транзакций подтверждений об успешной фиксации от каждого из диспетчеров ресурсов;

Пример: сегментирование данных в web-системах



Пример: программно-аппаратная архитектура Google (Google BigTable / HBase)



СУБД SQL Server 2025 – Создание распределённых баз данных.

Секционирование в СУБД SQL Server

- предпочтительным способом локального секционирования (в рамках одной базы данных) является определение секционированных таблиц и индексов (*partitioned tables and indexes*);
- секционированные представления поддерживают объединение горизонтально секционированных данных, распределённых между базовыми таблицами, находящимися на одном или нескольких серверах;
 - локальное / распределенное секционированное представление (*local / distributed partitioned views*);
 - федеративные серверы баз данных (*federated database servers*);
- поддерживается несколько режимов репликации для различных вариантов использования.

Проектирование распределенных секционированных представлений

- при проектировании набора распределенных секционированных представлений необходимо выполнить следующие процедуры:
 - определить шаблон инструкций SQL, выполняемых приложением;
 - определить, каким образом таблицы связаны друг с другом (через внешние ключи или неявно через столбцы, используемые в соединениях);
 - сопоставить частоту инструкций SQL с секциями, определенными при анализе связей;
 - определить правила маршрутизации инструкций SQL;
- цель проектирования: локальное размещение большей части данных, необходимых для выполнения запросов, обрабатываемых конкретным сервером (хотя итоговый набор секций и таблиц может быть асимметричным);
- пример: секционирование данных по регионам.

Горизонтальное секционирование

- распределённое секционированное представление формируется как набор строк из отдельных таблиц-элементов, объединённых с помощью UNION ALL.
- ограничения для базовых таблиц:
 - не должны входить в запрос более одного раза;
 - не могут иметь индексов, созданных на любых вычисляемых столбцах;
 - все ограничения первичного ключа таблиц-элементов должны быть на одинаковом количестве столбцов;
 - должна быть одна и та же настройка дополнения ANSI (параметр конфигурации ANSI_PADDING);
- ограничения для списка выборки:
 - все столбцы в каждой таблице-элементе должны быть включены в список выборки;
 - на столбцы нельзя ссылаться в списке выборки больше одного раза;
 - столбцы должны быть одного типа, идти в одном и том же порядке и встречаться только один раз.

```
SELECT
    <select_list1>
FROM T1
UNION ALL
SELECT
    <select_list2>
FROM T2
UNION ALL
...
SELECT
    <select_listn>
FROM Tn;
```

Ограничения для столбца секционирования

- для секционирования может использоваться только один столбец (столбец секционирования – *partitioning column*), который должен присутствовать в каждой таблице-элементе и находиться в одинаковом порядковом расположении в списках выборки всех инструкций SELECT в представлении (возможно использование SELECT *);
- столбец секционирования:
 - должен входить в первичный ключ;
 - не должен быть определён как вычисляемый, с автоматическим приращением, со значением по умолчанию или типа timestamp;
 - должен иметь единственное ограничение CHECK с операторами [IN, BETWEEN, AND, OR, <, <=, >, >=, =];
- диапазоны значений ключа секционирования, определённые во всех таблицах, не должны пересекаться.

Создание распределенных секционированных представлений: пример

```
-- On Server1:
CREATE TABLE Customers_33
  (CustomerID INTEGER PRIMARY KEY
    CHECK (CustomerID BETWEEN 1 AND 32999),
  ... -- Additional column definitions)

-- On Server2:
CREATE TABLE Customers_66
  (CustomerID INTEGER PRIMARY KEY
    CHECK (CustomerID BETWEEN 33000 AND 65999),
  ... -- Additional column definitions)

-- On Server3:
CREATE TABLE Customers_99
  (CustomerID INTEGER PRIMARY KEY
    CHECK (CustomerID BETWEEN 66000 AND 99999),
  ... -- Additional column definitions)

-- On EACH server:
CREATE VIEW Customers AS
  SELECT * FROM CompanyDatabase.TableOwner.Customers_33
  UNION ALL
  SELECT * FROM Server2.CompanyDatabase.TableOwner.Customers_66
  UNION ALL
  SELECT * FROM Server3.CompanyDatabase.TableOwner.Customers_99
```


Изменение данных в распределённых секционированных представлениях

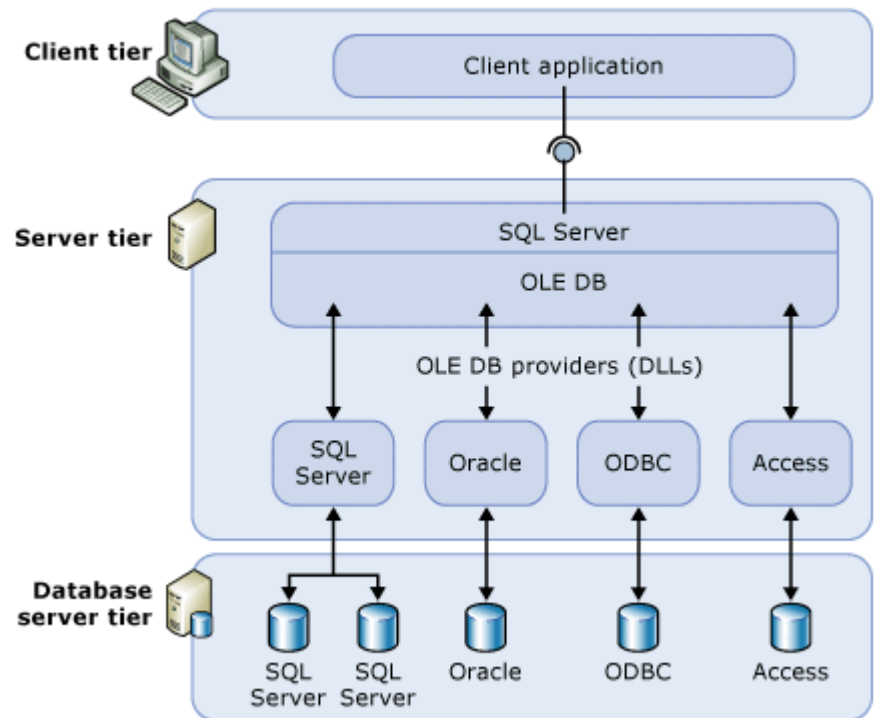
- в инструкцию INSERT должны включаться все столбцы (кроме столбцов с автоматическим приращением), даже если в базовой таблице значением столбца может быть NULL или если для него определено ограничение DEFAULT, также при вставке или обновлении не могут указываться значения по умолчанию (с помощью ключевого слова DEFAULT);
- значение столбца секционирования должно соответствовать одному из заданных ограничений;
- изменение данных через распределённое секционированное представление невозможно, если:
 - какая-либо из базовых таблиц содержит столбец типа timestamp (кроме инструкции DELETE);
 - для какой-либо из базовых таблиц определён триггер или ограничение ON UPDATE CASCADE/SET NULL/SET DEFAULT or ON DELETE CASCADE/SET NULL/SET DEFAULT;
 - есть самосоединение с тем же самым представлением или с одной из базовых таблиц.

Репликация

- СУБД SQL Server поддерживает следующие механизмы репликации:
 - транзакционная репликация (*transactional replication*)
начинается с создания исходного моментального снимка объектов и данных базы данных публикации, последующие изменения данных и схемы на издателе обычно доставляются подписчику без задержек (практически в реальном времени), а изменения данных применяются на подписчике в том же порядке и в тех же рамках транзакций, в которых они выполнялись у издателя (в пределах публикации гарантируется согласованность транзакций);
 - репликация слиянием (*merge replication*);
 - репликация моментальных снимков (*snapshot replication*)
распространяет данные подписчикам целиком и точно в том виде, в котором они были представлены в момент синхронизации, и не контролирует обновления этих данных.

Распределенные запросы

- распределенные запросы используются для доступа к данным, внешним по отношению к экземпляру СУБД SQL Server и расположенным в возможно гетерогенных источниках данных:
 - в других экземплярах SQL Server;
 - в различных реляционных и нереляционных источниках данных, представляющих данные в табличных объектах, именуемых наборами строк (*rowset*);
- доступ к внешним данным осуществляется посредством поставщиков данных OLE DB (*OLEDB providers*).



Взаимодействие с поставщиками OLE DB

- поставщики OLE DB для различных источников данных могут в зависимости от реализации поддерживать следующие функции:
 - только открытие и чтение набора строк базовой таблицы;
 - выполнение инструкций SELECT, INSERT, UPDATE, DELETE;
 - выполнение транзакций;
 - сканирование индексов;
 - использование курсоров, в том числе для изменения и удаления данных;
 - использование параметризованных запросов;
- при обработке распределённых запросов экземпляр SQL Server также выполняет следующие действия:
 - подключение к источнику данных;
 - получение необходимых метаданных;
 - управление выполнением транзакций;
 - преобразование типов данных (между типами OLE DB и встроенными типами);
 - обработку ошибок;
 - управление правами доступа.

Выполнение и оптимизация распределённых запросов

- SQL Server максимально делегирует выполнение распределенного запроса поставщику данных в зависимости от поддерживаемых им возможностей;
- на степень делегирования в том числе влияют:
 - уровень диалекта (грамматики SQL-запросов – SQL Minimum, ODBC Core, SQL-92 Entry level, SQL Server) и дополнительные возможности синтаксиса запросов, поддерживаемые поставщиком;
 - совместимость параметров сопоставления строк (*collation*);
- при оптимизации плана выполнения запроса при наличии поддержки со стороны поставщика данных также могут использоваться:
 - сканирование индексов удалённых таблиц;
 - использование статистик удалённых таблиц;
 - выполнение параметризованных запросов;
 - информация об ограничениях, определённых в базовых таблицах.

Распределённые транзакции

- SQL Server обеспечивает выполнение и управление удалёнными (локальными для источника данных) и распределёнными транзакциями в том случае, если транзакции поддерживаются источником данных и поставщиком (в противном случае операции чтения/записи могут быть запрещены или ограничены);
- распределённая транзакция инициируется явно (инструкцией `BEGIN DISTRIBUTED TRANSACTION`) или неявно (выполнением распределённого запроса или удалённой хранимой процедуры внутри локальной транзакции);
- процедура завершения распределённой транзакции выполняется в соответствии с протоколом двухфазной фиксации, причём в качестве диспетчера транзакций может выступать координатор распределённых транзакций MS DTC (Distributed Transaction Coordinator) или любой другой диспетчер транзакций, поддерживающий спецификацию X/Open XA.

Связанный сервер

- связанным сервером (*linked server*) называется зарегистрированный на экземпляре СУБД SQL Server источник данных с указанием соответствующего поставщика OLE DB и параметров подключения;
- в инструкциях языка SQL (SELECT, INSERT, UPDATE, DELETE) к объектам связанного сервера можно обращаться путём указания их полного имени (*< linked-server >.<catalog>.<schema>.<object>*), включающего имя связанного сервера;
- параметры *catalog*, *schema* и *object_name* представляют собой ссылку на объект источника данных, представляемый как набор строк, и передаются поставщику OLE DB для идентификации конкретного объекта данных (для СУБД SQL Server параметр *catalog* соответствует базе данных);
- доступ к связанному серверу также может осуществляться следующими способами:
 - через команду OPENQUERY для выполнения сквозного запроса (*pass-through query*) на связанном сервере;
 - через инструкцию EXECUTE AT *Linked_server_name*, позволяющую выполнить произвольный динамически сформированный параметризованный запрос на удалённом сервере.

Операции со связанными серверами

- получение списка зарегистрированных серверов:
 - `sp_linkedservers`;
 - `sys.servers` (представление системного каталога);
- добавление связанного сервера:
 - `sp_addlinkedserver`;
- удаление связанного сервера:
 - `sp_dropserver`;
- добавление / удаление имени входа (сопоставление имени и пароля удалённого входа для заданного локального входа):
 - `sp_addlinkedsrvlogin`
 - `sp_droplinkedsrvlogin`.

```
EXEC sp_addlinkedserver  
    N'SEATTLESales', N'SQL Server'
```

```
EXECUTE sp_addlinkedserver  
    @server = N'S1_instance1',  
    @srvproduct = N'',  
    @provider = N'MSOLEDBSQL',  
    @datasrc = N'S1\instance1';
```

```
EXEC sp_addlinkedserver  
    @server = 'LONDON Mktg',  
    @srvproduct = 'Oracle',  
    @provider = 'MSDAORA',  
    @datasrc = 'MyServer'
```

```
EXEC sp_droplinkedsrvlogin  
    @rmtsrvname = 'ServerOne',  
    @locallogin = NULL
```

```
EXECUTE master.dbo.sp_addlinkedsrvlogin  
    @rmtsrvname = N'LinkServerName',  
    @locallogin = NULL,  
    @useself = N'False',  
    @rmtuser = N'myUser',  
    @rmtpassword = N'*****';
```


Нерегистрируемые серверы (специальные серверы, ad hoc servers)

- команды OPENROWSET и OPENDATASOURCE обеспечивают возможность доступа к удалённым источникам данных без регистрации связанного сервера;
- команда OPENDATASOURCE:
 - может использоваться там, где синтаксис Transact-SQL позволяет указать имя связанного сервера;
 - требует указания параметров подключения к источнику данных OLE DB (поставщика данных и строки подключения);
- команда OPENROWSET:
 - может применяться везде, где в инструкции языка Transact-SQL используется ссылка на таблицу или представление;
 - требует указания параметров подключения к источнику данных OLE DB (поставщика данных и строки подключения), а также имени объекта или запроса, которые должны формировать набор строк;
- команды OPENROWSET и OPENDATASOURCE не поддерживают передачу переменных языка Transact-SQL в качестве аргументов;
- команды OPENROWSET и OPENDATASOURCE рекомендуется использовать только для единичных обращений к источникам данных, так как они не предоставляют всех возможностей определений связанных серверов, а также при каждом использовании требуют указания всей информации, необходимой для подключения (в том числе паролей).

Нерегистрируемые серверы: пример

```
SELECT GroupName, Name, DepartmentID
FROM OPENDATASOURCE('MSOLEDBSQL',
    'Server=Seattle1;Database=AdventureWorks2022;TrustServerCertificate=
    Yes;Trusted_Connection=Yes;').HumanResources.Department
ORDER BY GroupName, Name;
```

```
SELECT d.* FROM OPENROWSET(
    'MSOLEDBSQL',
    'Server=Seattle1;Trusted_Connection=yes;',
    AdventureWorks2022.HumanResources.Department
) AS d;
```

```
SELECT a.*
FROM OPENROWSET(
    'MSOLEDBSQL', 'Server=Seattle1;Trusted_Connection=yes;',
    'SELECT GroupName, Name, DepartmentID
        FROM AdventureWorks2022.HumanResources.Department
        ORDER BY GroupName, Name'
) AS a;
```

Сквозные запросы

- команда **OPENQUERY** позволяет выполнить передаваемый запрос к указанному связанному серверу:
 - на команду **OPENQUERY** как на имя таблицы можно ссылаться из предложения **FROM** инструкции **SELECT**, а также в инструкциях **INSERT**, **UPDATE** или **DELETE**;
 - в качестве аргументов нельзя использовать переменные;
 - команда **OPENQUERY** не может быть использована для выполнения расширенных хранимых процедур на связанном сервере.

```
SELECT * FROM OPENQUERY (OracleSvr,  
    'SELECT name FROM joe.titles WHERE name = ''NewTitle''');
```

```
INSERT OPENQUERY (OracleSvr, 'SELECT name FROM joe.titles')  
    VALUES ('NewTitle');
```

```
UPDATE OPENQUERY (OracleSvr, 'SELECT name FROM joe.titles WHERE id = 101')  
    SET name = 'ADifferentName';
```

```
DELETE OPENQUERY (OracleSvr,  
    'SELECT name FROM joe.titles WHERE name = ''NewTitle''');
```

Вопросы к экзамену

- Распределение данных в СУБД. Горизонтальная и вертикальная фрагментация.
- Распределение данных в СУБД: репликация. Согласованность реплик и распространение изменений. Репликация в СУБД SQL Server.
- Параллелизм в СУБД. Параллельные СУБД.
- Распределённые СУБД.
- Согласованность в распределённых системах. Протокол двухфазной фиксации транзакций. Распределённые транзакции в СУБД SQLServer.
- Горизонтальное секционирование в СУБД SQL Server. Проектирование и создание распределённых секционированных представлений.
- Распределённые запросы. Взаимодействие с поставщиками данных. Выполнение и оптимизация распределённых запросов.
- Связанные серверы. Нерегистрируемые серверы (OPENDATASOURCE, OPENROWSET). Сквозные запросы (OPENQUERY).

Лабораторная работа №13.

Создание распределенных баз данных на основе секционированных представлений

1. Создать две базы данных на одном экземпляре СУБД SQL Server.
2. Создать в базах данных п.1. горизонтально фрагментированные таблицы.
3. Создать секционированные представления, обеспечивающие работу с данными таблиц (выборку, вставку, изменение, удаление).

Лабораторная работа №14.

Создание вертикально фрагментированных таблиц средствами СУБД SQL Server

1. Создать в базах данных пункта 1 задания 13 таблицы, содержащие вертикально фрагментированные данные.
2. Создать необходимые элементы базы данных (представления, триггеры), обеспечивающие работу с данными вертикально фрагментированных таблиц (выборку, вставку, изменение, удаление).

Лабораторная работа №15.

Создание распределенных баз данных со связанными таблицами средствами СУБД SQL Server

1. Создать в базах данных пункта 1 задания 13 связанные таблицы.
2. Создать необходимые элементы базы данных (представления, триггеры), обеспечивающие работу с данными связанных таблиц (выборку, вставку, изменение, удаление).